

Java OOP Study Notes: Abstraction

1. What is Abstraction?

Abstraction is the process of hiding implementation details and showing only the essential features to the user.

Real-World Analogy

Object	What You See (Abstract)	What's Hidden (Implementation)
Car	Steering, pedals, gear	Engine mechanics, fuel injection
ATM	Screen, keypad, card slot	Cash counting, security, banking network
TV Remote	Buttons	Infrared signals, encoding
Coffee Machine	Start button	Water heating, pressure systems

You don't need to know HOW it works internally — just WHAT it does.

Two Ways to Achieve Abstraction in Java

Abstraction

/

\

/

\

Abstract Class

Interface

(0-100% abstract)

(100% abstract*)

* Before Java 8, interfaces were 100% abstract

* Now can have default/static methods

2. Abstract Class

An **abstract class** is a class that cannot be instantiated and may contain abstract methods (without body) and concrete methods (with body).

Syntax

```
abstract class ClassName {  
    // Abstract method (no body)  
    abstract void abstractMethod();  
  
    // Concrete method (has body)  
    void concreteMethod() {  
        System.out.println("This has implementation");  
    }  
}
```

Key Rules

Rule	Description
Cannot be instantiated	<code>new AbstractClass()</code> → ERROR
Can have constructors	Called via <code>super()</code> from child
Can have any access modifiers	public, protected, private, default
Can have instance variables	Both final and non-final
Can have static members	Static methods and variables
Child must implement abstract methods	Or be abstract itself

Basic Example

```
abstract class Animal {
    String name;

    // Constructor
    Animal(String name) {
        this.name = name;
        System.out.println("Animal constructor called");
    }

    // Abstract method - NO body
    abstract void makeSound();

    // Concrete method - HAS body
    void sleep() {
        System.out.println(name + " is sleeping...");
    }
}

class Dog extends Animal {

    Dog(String name) {
        super(name);
    }

    // MUST implement abstract method
    @Override
    void makeSound() {
        System.out.println(name + " says: Woof! Woof!");
    }
}

class Cat extends Animal {

    Cat(String name) {
        super(name);
    }

    @Override
```

```
void makeSound() {
    System.out.println(name + " says: Meow!");
}

public class Main {
    public static void main(String[] args) {
        // Animal a = new Animal("Test"); // ERROR! Cannot instantiate abstract class

        Animal dog = new Dog("Buddy");
        Animal cat = new Cat("Whiskers");

        dog.makeSound(); // Buddy says: Woof! Woof!
        dog.sleep();      // Buddy is sleeping...

        cat.makeSound(); // Whiskers says: Meow!
        cat.sleep();      // Whiskers is sleeping...
    }
}
```

Output:

```
Animal constructor called
Animal constructor called
Buddy says: Woof! Woof!
Buddy is sleeping...
Whiskers says: Meow!
Whiskers is sleeping...
```

3. Abstract Methods

Abstract methods are declared without an implementation (no body). They define WHAT must be done, not HOW.

Syntax

```
// Abstract method - ends with semicolon, no curly braces  
abstract returnType methodName(parameters);
```

Rules for Abstract Methods

Rule	Description
No body	Just declaration, ends with <code>;</code>
Must be in abstract class	Cannot have abstract method in concrete class
Cannot be <code>final</code>	Final methods cannot be overridden
Cannot be <code>private</code>	Must be accessible to subclasses
Cannot be <code>static</code>	Static methods belong to class, not overridden
Child must implement	Unless child is also abstract

Example: Shape Hierarchy

```
abstract class Shape {
    String color;

    Shape(String color) {
        this.color = color;
    }

    // Abstract methods - children MUST implement
    abstract double getArea();
    abstract double getPerimeter();
    abstract void draw();

    // Concrete method - children inherit as-is
    void displayColor() {
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape {
    double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    @Override
    void draw() {
```

```
        System.out.println("Drawing a " + color + " circle with radius " + radius);
    }
}

class Rectangle extends Shape {
    double width, height;

    Rectangle(String color, double width, double height) {
        super(color);
        this.width = width;
        this.height = height;
    }

    @Override
    double getArea() {
        return width * height;
    }

    @Override
    double getPerimeter() {
        return 2 * (width + height);
    }

    @Override
    void draw() {
        System.out.println("Drawing a " + color + " rectangle " + width + "x" + height);
    }
}
```

4. Concrete Methods in Abstract Class

Abstract classes can have **concrete methods** (methods with full implementation) that provide common functionality to all subclasses.

Why Mix Abstract and Concrete?

Abstract Class

- └ Abstract Methods → Force children to implement (contract)
- └ Concrete Methods → Share common code (reuse)

Practical Example

```
abstract class Employee {
    protected String name;
    protected String id;
    protected double baseSalary;

    Employee(String name, String id, double baseSalary) {
        this.name = name;
        this.id = id;
        this.baseSalary = baseSalary;
    }

    // =====
    // ABSTRACT METHODS - Each employee type calculates differently
    // =====
    abstract double calculateSalary();
    abstract String getRole();

    // =====
    // CONCRETE METHODS - Same for all employees
    // =====

    void displayBasicInfo() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Role: " + getRole());
    }

    void printPaySlip() {
        System.out.println("=====");
        System.out.println("||          PAY SLIP          ||");
        System.out.println("=====");
        displayBasicInfo();
        System.out.println("Base Salary: $" + baseSalary);
        System.out.println("Total Salary: $" + calculateSalary());
        System.out.println("=====");
    }

    // Concrete method using abstract method
```

```

    double getAnnualSalary() {
        return calculateSalary() * 12;
    }

    // Static utility method
    static void printCompanyHeader() {
        System.out.println("*** ABC Corporation ***");
    }
}

class Developer extends Employee {
    private int projectBonus;

    Developer(String name, String id, double baseSalary, int projectBonus) {
        super(name, id, baseSalary);
        this.projectBonus = projectBonus;
    }

    @Override
    double calculateSalary() {
        return baseSalary + projectBonus;
    }

    @Override
    String getRole() {
        return "Software Developer";
    }
}

class Manager extends Employee {
    private double teamBonus;
    private int teamSize;

    Manager(String name, String id, double baseSalary, int teamSize) {
        super(name, id, baseSalary);
        this.teamSize = teamSize;
        this.teamBonus = teamSize * 500; // $500 per team member
    }

    @Override

```

```

    double calculateSalary() {
        return baseSalary + teamBonus;
    }

    @Override
    String getRole() {
        return "Team Manager";
    }
}

class Intern extends Employee {

    Intern(String name, String id) {
        super(name, id, 20000); // Fixed salary for interns
    }

    @Override
    double calculateSalary() {
        return baseSalary; // No bonuses
    }

    @Override
    String getRole() {
        return "Intern";
    }
}

public class Main {
    public static void main(String[] args) {
        Employee.printCompanyHeader();

        Employee[] employees = {
            new Developer("Aslam", "DEV001", 60000, 5000),
            new Manager("Rahim", "MGR001", 80000, 5),
            new Intern("Karim", "INT001")
        };

        for (Employee emp : employees) {
            emp.printPaySlip(); // Concrete method calls abstract methods
            System.out.println("Annual: $" + emp.getAnnualSalary());
        }
    }
}

```

```
        System.out.println();  
    }  
}  
}
```

Output:

*** ABC Corporation ***

PAY SLIP

ID: DEV001

Name: Aslam

Role: Software Developer

Base Salary: \$60000.0

Total Salary: \$65000.0

Annual: \$780000.0

PAY SLIP

ID: MGR001

Name: Rahim

Role: Team Manager

Base Salary: \$80000.0

Total Salary: \$82500.0

Annual: \$990000.0

PAY SLIP

ID: INT001

Name: Karim

Role: Intern

Base Salary: \$20000.0

Total Salary: \$20000.0

Annual: \$240000.0

5. Interface

An **interface** is a completely abstract type that defines a contract — a set of methods that implementing classes MUST provide.

Syntax

```
interface InterfaceName {  
    // Constants (implicitly public static final)  
    int MAX_VALUE = 100;  
  
    // Abstract methods (implicitly public abstract)  
    void method1();  
    int method2(String param);  
  
    // Default method (Java 8+)  
    default void defaultMethod() {  
        System.out.println("Default implementation");  
    }  
  
    // Static method (Java 8+)  
    static void staticMethod() {  
        System.out.println("Static in interface");  
    }  
}
```

Key Rules

Rule	Description
All methods are <code>public abstract</code>	By default (before Java 8)
All variables are <code>public static final</code>	Constants only
Cannot have constructors	No instance creation
Cannot have instance variables	Only static final constants
A class <code>implements</code> interface	Not <code>extends</code>
Can implement multiple interfaces	Multiple inheritance!

Basic Example

```
interface Drawable {
    void draw();
}

interface Resizable {
    void resize(double factor);
}

class Circle implements Drawable, Resizable {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public void draw() {
        System.out.println("Drawing circle with radius: " + radius);
    }

    @Override
    public void resize(double factor) {
        radius *= factor;
        System.out.println("Resized to radius: " + radius);
    }
}

public class Main {
    public static void main(String[] args) {
        Circle c = new Circle(5);
        c.draw();
        c.resize(2);
        c.draw();

        // Interface as reference type
        Drawable d = new Circle(10);
        d.draw();
        // d.resize(2); // ERROR! Drawable doesn't have resize()
    }
}
```

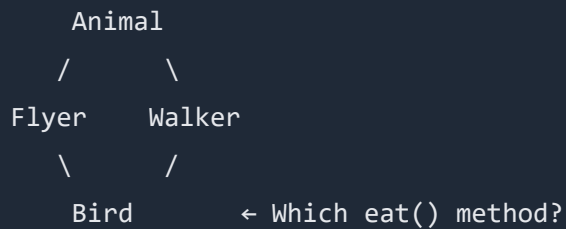


```
}  
}
```

6. Multiple Inheritance via Interfaces

Java doesn't support multiple inheritance with classes (Diamond Problem), but DOES support it with interfaces.

The Diamond Problem (Why Classes Can't)



Solution: Interfaces

```
interface Flyable {
    void fly();

    default void takeOff() {
        System.out.println("Taking off...");
    }
}

interface Swimmable {
    void swim();

    default void dive() {
        System.out.println("Diving...");
    }
}

interface Walkable {
    void walk();
}

// A class can implement MULTIPLE interfaces
class Duck implements Flyable, Swimmable, Walkable {

    @Override
    public void fly() {
        System.out.println("Duck flying through the air");
    }

    @Override
    public void swim() {
        System.out.println("Duck swimming in the pond");
    }

    @Override
    public void walk() {
        System.out.println("Duck waddling on land");
    }
}
```

```
class Penguin implements Swimmable, Walkable {
    // Penguins can't fly!

    @Override
    public void swim() {
        System.out.println("Penguin swimming gracefully");
    }

    @Override
    public void walk() {
        System.out.println("Penguin waddling cutely");
    }
}

class Fish implements Swimmable {

    @Override
    public void swim() {
        System.out.println("Fish swimming fast");
    }
}

public class Main {
    public static void main(String[] args) {
        Duck duck = new Duck();
        duck.fly();
        duck.swim();
        duck.walk();
        duck.takeOff(); // Default method from Flyable
        duck.dive();    // Default method from Swimmable

        System.out.println();

        // Polymorphism with interfaces
        Swimmable[] swimmers = { new Duck(), new Penguin(), new Fish() };

        for (Swimmable s : swimmers) {
            s.swim();
        }
    }
}
```

```
}  
}
```

Output:

```
Duck flying through the air  
Duck swimming in the pond  
Duck waddling on land  
Taking off...  
Diving...  
  
Duck swimming in the pond  
Penguin swimming gracefully  
Fish swimming fast
```

Interface Inheritance

Interfaces can extend other interfaces:

```
interface Animal {
    void eat();
}

interface Mammal extends Animal {
    void breathe();
}

interface Pet extends Mammal {
    void play();
}

// Dog must implement ALL methods from the chain
class Dog implements Pet {
    @Override
    public void eat() {
        System.out.println("Dog eating");
    }

    @Override
    public void breathe() {
        System.out.println("Dog breathing");
    }

    @Override
    public void play() {
        System.out.println("Dog playing fetch");
    }
}
```

Interface Extending Multiple Interfaces

```
interface A {  
    void methodA();  
}  
  
interface B {  
    void methodB();  
}  
  
interface C extends A, B { // Interface can extend multiple interfaces!  
    void methodC();  
}  
  
class MyClass implements C {  
    @Override  
    public void methodA() { }  
  
    @Override  
    public void methodB() { }  
  
    @Override  
    public void methodC() { }  
}
```

7. Default Methods (Java 8+)

Default methods allow interfaces to have methods with implementation. This enables adding new methods to interfaces without breaking existing implementations.

Syntax

```
interface MyInterface {  
    default void defaultMethod() {  
        // Has a body!  
        System.out.println("Default implementation");  
    }  
}
```

Why Default Methods?

Before Java 8, adding a method to an interface would break ALL implementing classes. Default methods solve this:

```
// Original interface  
interface Vehicle {  
    void start();  
    void stop();  
}  
  
// 100 classes implement Vehicle...  
  
// Later, we want to add a new method  
interface Vehicle {  
    void start();  
    void stop();  
  
    // Without default: ALL 100 classes must be updated!  
    // With default: Existing classes still work  
    default void honk() {  
        System.out.println("Beep beep!");  
    }  
}
```

Example

```
interface Printable {  
    void print();  
  
    // Default method with implementation  
    default void printMultiple(int times) {  
        for (int i = 0; i < times; i++) {  
            print();  
        }  
    }  
  
    default void printWithBorder() {  
        System.out.println("=====");  
        print();  
        System.out.println("=====");  
    }  
}  
  
class Document implements Printable {  
    private String content;  
  
    Document(String content) {  
        this.content = content;  
    }  
  
    @Override  
    public void print() {  
        System.out.println("Document: " + content);  
    }  
  
    // Can override default method  
    @Override  
    public void printWithBorder() {  
        System.out.println("*** DOCUMENT ***");  
        print();  
        System.out.println("*****");  
    }  
}
```



```

class Image implements Printable {
    private String name;

    Image(String name) {
        this.name = name;
    }

    @Override
    public void print() {
        System.out.println("[Image: " + name + "]");
    }

    // Uses default printWithBorder() as-is
}

public class Main {
    public static void main(String[] args) {
        Document doc = new Document("Hello World");
        doc.print();
        doc.printMultiple(3);
        doc.printWithBorder(); // Overridden version

        System.out.println();

        Image img = new Image("photo.jpg");
        img.print();
        img.printWithBorder(); // Default version
    }
}

```

Resolving Diamond Problem with Default Methods

When a class implements multiple interfaces with the same default method:

```
interface A {
    default void greet() {
        System.out.println("Hello from A");
    }
}

interface B {
    default void greet() {
        System.out.println("Hello from B");
    }
}

class C implements A, B {
    // MUST override to resolve conflict!
    @Override
    public void greet() {
        // Option 1: Provide own implementation
        System.out.println("Hello from C");

        // Option 2: Call specific interface's method
        A.super.greet(); // Calls A's version
        B.super.greet(); // Calls B's version
    }
}
```

8. Static Methods in Interface (Java 8+)

Interfaces can have **static methods** with implementation. These belong to the interface itself, not implementing classes.

Syntax

```
interface MyInterface {  
    static void staticMethod() {  
        System.out.println("Static method in interface");  
    }  
}  
  
// Called using interface name  
MyInterface.staticMethod();
```

Example: Utility Methods

```
interface StringUtils {

    // Abstract method
    String process(String input);

    // Static utility methods
    static boolean isEmpty(String str) {
        return str == null || str.trim().isEmpty();
    }

    static String capitalize(String str) {
        if (isEmpty(str)) return str;
        return str.substring(0, 1).toUpperCase() + str.substring(1).toLowerCase();
    }

    static String reverse(String str) {
        if (isEmpty(str)) return str;
        return new StringBuilder(str).reverse().toString();
    }

    static int countWords(String str) {
        if (isEmpty(str)) return 0;
        return str.trim().split("\\s+").length;
    }
}

class UpperCaseProcessor implements StringUtils {
    @Override
    public String process(String input) {
        return input.toUpperCase();
    }
}

public class Main {
    public static void main(String[] args) {
        // Call static methods directly on interface
        System.out.println(StringUtils.isEmpty(""));           // true
        System.out.println(StringUtils.isEmpty("hello"));      // false
    }
}
```

```

        System.out.println(StringUtils.capitalize("jAVA"));    // Java
        System.out.println(StringUtils.reverse("Hello"));      // olleH
        System.out.println(StringUtils.countWords("Hello World")); // 2

        // Instance method through implementing class
        StringUtils processor = new UpperCaseProcessor();
        System.out.println(processor.process("hello"));         // HELLO

        // Note: Cannot call static method through instance
        // processor.isEmpty(""); // This works but is bad practice
    }
}

```

Static Methods Are NOT Inherited

```

interface A {
    static void show() {
        System.out.println("A's static method");
    }
}

class B implements A {
    // B does NOT inherit show()
}

public class Main {
    public static void main(String[] args) {
        A.show();    // OK - Call on interface
        // B.show(); // ERROR - B doesn't have show()
    }
}

```

9. Private Methods in Interface (Java 9+)

Java 9 added **private methods** in interfaces to share code between default methods.

```
interface Logger {

    default void logInfo(String message) {
        log("INFO", message);
    }

    default void logWarning(String message) {
        log("WARNING", message);
    }

    default void logError(String message) {
        log("ERROR", message);
    }

    // Private helper method - not visible to implementing classes
    private void log(String level, String message) {
        System.out.println "[" + level + "] " +
            java.time.LocalDateTime.now() + ": " + message);
    }

    // Private static helper
    private static String formatMessage(String msg) {
        return msg.trim().toUpperCase();
    }
}
```

10. Abstract Class vs Interface

Comparison Table

Feature	Abstract Class	Interface
Keyword	<code>abstract class</code>	<code>interface</code>
Inheritance	<code>extends</code> (single)	<code>implements</code> (multiple)
Methods	Abstract + Concrete	Abstract + Default + Static
Variables	Any type	Only <code>public static final</code>
Constructors	Yes	No
Access Modifiers	Any	Only <code>public</code> (methods)
Multiple Inheritance	No	Yes
When to Use	IS-A relationship with shared code	CAN-DO capability/contract

Visual Comparison

ABSTRACT CLASS

- ✓ Instance variables (any type)
- ✓ Constructors
- ✓ Abstract methods
- ✓ Concrete methods
- ✓ Static methods
- ✓ Any access modifier (public, protected, private)
- X Multiple inheritance

Use when: Classes share common code AND have IS-A relation

INTERFACE

- X Instance variables (only constants)
- X Constructors
- ✓ Abstract methods
- ✓ Default methods (Java 8+)
- ✓ Static methods (Java 8+)
- ✓ Private methods (Java 9+)
- ✓ Multiple inheritance

Use when: Defining a contract/capability (CAN-DO)

When to Use Which?

Use ABSTRACT CLASS when:

- └─ You want to share code among related classes
- └─ Classes have common state (instance variables)
- └─ You need non-public members
- └─ You need constructors
- └─ There's a clear IS-A relationship

Use INTERFACE when:

- └─ You want to define a contract (what, not how)
- └─ Unrelated classes need same capability
- └─ You need multiple inheritance
- └─ You're defining a type for polymorphism
- └─ There's a CAN-DO relationship

Example: Both Together

```
// Interface - defines capabilities
interface Playable {
    void play();
    void pause();
    void stop();

    default void restart() {
        stop();
        play();
    }
}

interface Recordable {
    void record();
    void saveRecording(String filename);
}

// Abstract class - defines common structure
abstract class MediaPlayer implements Playable {
    protected String currentMedia;
    protected boolean isPlaying;

    MediaPlayer() {
        this.isPlaying = false;
    }

    // Concrete method
    void loadMedia(String media) {
        this.currentMedia = media;
        System.out.println("Loaded: " + media);
    }

    // Abstract method - each player implements differently
    abstract void displayInfo();

    // Implement interface methods
    @Override
    public void play() {
```

```

        if (currentMedia != null) {
            isPlaying = true;
            System.out.println("Playing: " + currentMedia);
        }
    }

    @Override
    public void pause() {
        isPlaying = false;
        System.out.println("Paused");
    }

    @Override
    public void stop() {
        isPlaying = false;
        currentMedia = null;
        System.out.println("Stopped");
    }
}

// Concrete class - extends abstract + implements additional interface
class VideoPlayer extends MediaPlayer implements Recordable {
    private String resolution;

    VideoPlayer(String resolution) {
        super();
        this.resolution = resolution;
    }

    @Override
    void displayInfo() {
        System.out.println("Video Player - Resolution: " + resolution);
        System.out.println("Current: " + (currentMedia != null ? currentMedia : "None"));
        System.out.println("Status: " + (isPlaying ? "Playing" : "Stopped"));
    }

    @Override
    public void record() {
        System.out.println("Recording video at " + resolution);
    }
}

```

```

@Override
public void saveRecording(String filename) {
    System.out.println("Saved recording to: " + filename);
}
}

class AudioPlayer extends MediaPlayer {
    private int volume;

    AudioPlayer() {
        super();
        this.volume = 50;
    }

    @Override
    void displayInfo() {
        System.out.println("Audio Player - Volume: " + volume);
        System.out.println("Current: " + (currentMedia != null ? currentMedia : "None"));
    }

    void setVolume(int volume) {
        this.volume = Math.max(0, Math.min(100, volume));
        System.out.println("Volume set to: " + this.volume);
    }
}

public class Main {
    public static void main(String[] args) {
        VideoPlayer video = new VideoPlayer("1080p");
        video.loadMedia("movie.mp4");
        video.displayInfo();
        video.play();
        video.record();
        video.saveRecording("recording.mp4");
        video.restart(); // Default method from interface

        System.out.println();

        AudioPlayer audio = new AudioPlayer();
    }
}

```

```

        audio.loadMedia("song.mp3");
        audio.displayInfo();
        audio.setVolume(75);
        audio.play();

        System.out.println();

        // Polymorphism with interface
        Playable[] players = { video, audio };
        System.out.println("=== Stopping all players ===");
        for (Playable p : players) {
            p.stop();
        }
    }
}

```

Output:

```

Loaded: movie.mp4
Video Player - Resolution: 1080p
Current: movie.mp4
Status: Stopped
Playing: movie.mp4
Recording video at 1080p
Saved recording to: recording.mp4
Stopped
Playing: movie.mp4

Loaded: song.mp3
Audio Player - Volume: 50
Current: song.mp3
Volume set to: 75
Playing: song.mp3

=== Stopping all players ===
Stopped
Stopped

```

11. Complete Abstraction Example

```
// =====  
// INTERFACES - Define capabilities  
// =====  
  
interface Taxable {  
    double TAX_RATE = 0.1; // 10% - public static final  
    double calculateTax();  
}  
  
interface Discountable {  
    double applyDiscount(double percentage);  
}  
  
interface Printable {  
    void printDetails();  
  
    default void printSummary() {  
        System.out.println("--- Summary ---");  
        printDetails();  
        System.out.println("-----");  
    }  
  
    static void printHeader(String title) {  
        System.out.println("=====");  
        System.out.printf("|| %-36s ||%n", title);  
        System.out.println("=====");  
    }  
}  
  
// =====  
// ABSTRACT CLASS - Common product structure  
// =====  
  
abstract class Product implements Printable {  
    protected String id;  
    protected String name;  
    protected double price;  
  
    private static int counter = 0;
```

```

    Product(String name, double price) {
        this.id = "PROD-" + (++counter);
        this.name = name;
        this.price = price;
    }

    // Abstract methods
    abstract String getCategory();
    abstract double getFinalPrice();

    // Concrete methods
    String getId() { return id; }
    String getName() { return name; }
    double getPrice() { return price; }

    @Override
    public void printDetails() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Category: " + getCategory());
        System.out.println("Base Price: $" + price);
        System.out.println("Final Price: $" + getFinalPrice());
    }
}

// =====
// CONCRETE CLASSES
// =====

class Electronics extends Product implements Taxable {
    private String brand;
    private int warrantyYears;

    Electronics(String name, double price, String brand, int warrantyYears) {
        super(name, price);
        this.brand = brand;
        this.warrantyYears = warrantyYears;
    }

    @Override
    String getCategory() {

```

```

        return "Electronics";
    }

    @Override
    public double calculateTax() {
        return price * TAX_RATE;
    }

    @Override
    double getFinalPrice() {
        return price + calculateTax();
    }

    @Override
    public void printDetails() {
        super.printDetails();
        System.out.println("Brand: " + brand);
        System.out.println("Warranty: " + warrantyYears + " years");
        System.out.println("Tax: $" + calculateTax());
    }
}

class Clothing extends Product implements Discountable {
    private String size;
    private String color;
    private double discountPercentage;

    Clothing(String name, double price, String size, String color) {
        super(name, price);
        this.size = size;
        this.color = color;
        this.discountPercentage = 0;
    }

    @Override
    String getCategory() {
        return "Clothing";
    }

    @Override

```



```

    public double applyDiscount(double percentage) {
        this.discountPercentage = Math.min(percentage, 50); // Max 50%
        return getFinalPrice();
    }

    @Override
    double getFinalPrice() {
        return price * (1 - discountPercentage / 100);
    }

    @Override
    public void printDetails() {
        super.printDetails();
        System.out.println("Size: " + size);
        System.out.println("Color: " + color);
        if (discountPercentage > 0) {
            System.out.println("Discount: " + discountPercentage + "%");
        }
    }
}

class Grocery extends Product implements Taxable, Discountable {
    private String expiryDate;
    private boolean isOrganic;
    private double discountPercentage;

    Grocery(String name, double price, String expiryDate, boolean isOrganic) {
        super(name, price);
        this.expiryDate = expiryDate;
        this.isOrganic = isOrganic;
        this.discountPercentage = 0;
    }

    @Override
    String getCategory() {
        return isOrganic ? "Organic Grocery" : "Grocery";
    }

    @Override
    public double calculateTax() {

```

```

        return isOrganic ? price * 0.05 : price * TAX_RATE; // Lower tax for organic
    }

    @Override
    public double applyDiscount(double percentage) {
        this.discountPercentage = Math.min(percentage, 20); // Max 20% for grocery
        return getFinalPrice();
    }

    @Override
    double getFinalPrice() {
        double discounted = price * (1 - discountPercentage / 100);
        return discounted + calculateTax();
    }

    @Override
    public void printDetails() {
        super.printDetails();
        System.out.println("Expiry: " + expiryDate);
        System.out.println("Organic: " + (isOrganic ? "Yes" : "No"));
        System.out.println("Tax: $" + calculateTax());
        if (discountPercentage > 0) {
            System.out.println("Discount: " + discountPercentage + "%");
        }
    }
}

// =====
// MAIN CLASS
// =====
public class Main {
    public static void main(String[] args) {

        // Create products
        Product[] products = {
            new Electronics("Laptop", 999.99, "Dell", 2),
            new Clothing("T-Shirt", 29.99, "M", "Blue"),
            new Grocery("Apples", 5.99, "2025-03-01", true),
            new Grocery("Bread", 2.99, "2025-02-28", false)
        };
    }
}

```

```
// Print all products (polymorphism through abstract class)
Printable.printHeader("PRODUCT CATALOG");

for (Product p : products) {
    System.out.println();
    p.printSummary(); // Default method from interface
}

// Apply discounts where applicable
System.out.println("\n=== Applying Discounts ===\n");

for (Product p : products) {
    if (p instanceof Discountable) {
        System.out.println("Applying 15% discount to: " + p.getName());
        double newPrice = ((Discountable) p).applyDiscount(15);
        System.out.println("New price: $" + newPrice);
        System.out.println();
    }
}

// Calculate total tax
System.out.println("=== Tax Summary ===\n");
double totalTax = 0;

for (Product p : products) {
    if (p instanceof Taxable) {
        double tax = ((Taxable) p).calculateTax();
        totalTax += tax;
        System.out.println(p.getName() + " tax: $" + tax);
    }
}

System.out.println("\nTotal Tax: $" + totalTax);

// Final prices
System.out.println("\n=== Final Prices ===\n");
double grandTotal = 0;

for (Product p : products) {
    double finalPrice = p.getFinalPrice();
```

```
        grandTotal += finalPrice;
        System.out.printf("%-15s $%.2f%n", p.getName(), finalPrice);
    }
    System.out.println("-----");
    System.out.printf("%-15s $%.2f%n", "GRAND TOTAL:", grandTotal);
}
}
```

12. Key Points to Remember

Abstract Class

1. Declared with `abstract` keyword
2. Cannot be instantiated directly
3. Can have abstract + concrete methods
4. Can have constructors, instance variables
5. Supports single inheritance only
6. Use for IS-A relationship with shared code

Interface

1. Declared with `interface` keyword
2. All methods are `public abstract` by default
3. All variables are `public static final`
4. No constructors
5. Supports multiple inheritance
6. Use for CAN-DO capabilities/contracts

Default Methods (Java 8+)

7. Allow implementation in interfaces
8. Enable adding methods without breaking existing code
9. Can be overridden by implementing classes
10. Resolve conflicts with `InterfaceName.super.method()`

Static Methods (Java 8+)

11. Belong to interface, not instances
12. Called using `InterfaceName.staticMethod()`
13. Not inherited by implementing classes

Best Practice

14. **Prefer interfaces** for defining types
15. **Use abstract classes** when you need to share code
16. **Combine both** for powerful, flexible designs

```
'; triggerDownload(docTitle.replace(/^[a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html;charset=utf-8'); }  
function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); //  
Force light mode for PDF (dark backgrounds print poorly) const wasDark =  
document.documentElement.classList.contains('dark'); if (wasDark)  
document.documentElement.classList.remove('dark'); const element =  
document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename:  
docTitle.replace(/^[a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale:  
1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' },  
pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if  
(wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid  
rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme:  
dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre  
code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !==  
'mermaid') { hljs.highlightElement(block); } }); updateHljsTheme(isDark); // Render mermaid diagrams try {  
mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const  
mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i <
```

```
mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const
code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const
container = document.createElement('div'); container.className = 'mermaid-container';
container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render
failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function
updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark;
document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */
window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const
isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark);
updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme:
isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) {} } });
```